



UNICA

UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI



sAifer Lab

Joint lab on Safety and Security of AI

Database

Angelo Sotgiu

angelo.sotgiu@unica.it

In questa lezione

- Creazione e gestione di un database relazionale
- Integrazione del database nell'applicazione web



Riepilogo...

- Nelle lezioni precedenti abbiamo costruito una semplice applicazione web per la gestione di una libreria contenente:
 - un livello "backend" che espone delle API
 - un livello "frontend" che fornisce un'interfaccia web
- Manca quindi una componente essenziale: il livello "dati"....
 - ...finora lo abbiamo simulato con una struttura dati fittizia
- Abbiamo bisogno di un **database** e di interfacce che consentano di eseguire delle operazioni su esso dal nostro backend
- Esistono tantissimi framework e librerie per la gestione dei database, noi useremo **SQLModel**
 - libreria scritta dallo stesso autore di FastAPI, fondamentalemente combina Pydantic e SQLAlchemy (una libreria popolarissima) e si integra con semplicità nelle webapp scritte con FastAPI



Creazione e inizializzazione database

- Per prima cosa, installiamo la libreria sqlalchemy
 - eseguiamo il comando `pip install sqlalchemy`
 - ricordiamoci anche di aggiungere sqlalchemy dentro il file "requirements.txt"
- Eliminiamo dalla cartella "data" il file "books.py" che abbiamo usato sinora per creare un finto elenco di libri, e creiamo nella stessa cartella un altro file "db.py" con questo codice:

```
from sqlalchemy import create_engine, SQLAlchemy, Session
```

```
sqlite_file_name = "app/data/database.db"  
sqlite_url = f"sqlite:/// {sqlite_file_name}"  
connect_args = {"check_same_thread": False}  
engine = create_engine(sqlite_url, connect_args=connect_args, echo=True)
```

Creazione e inizializzazione database

- Questo codice crea il motore del DB e verrà eseguito una sola volta nel momento in cui importeremo il modulo "data.db"
- Il DB verrà salvato nel file specificato e rimarrà persistente in memoria (anche quando l'app non è in esecuzione)
- Il DB ancora però non è inizializzato: serve un ulteriore passaggio. Per farlo, creiamo sempre nel file "db.py" un'altra funzione

```
def init_database() -> None:  
    SQLAlchemy.metadata.create_all(engine)
```

- È necessario richiamare la funzione `init_database()` a ogni avvio dell'applicazione

Creazione e inizializzazione database

- Per automatizzare questo processo, possiamo usare gli eventi *lifespan*: fanno in modo che in avvio e in chiusura dell'app vengano chiamate delle funzioni. Modifichiamo il "main.py" in questo modo:

```
from contextlib import asynccontextmanager
from data.db import init_database
```

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    # on start
    init_database()
    yield
    # on close
```

```
app = FastAPI(lifespan=lifespan)
```

Object-Relational Mapping (ORM)

- Per gestire il database da un linguaggio di programmazione (a oggetti) si fa tipicamente uso di ORM (Object-relational mapping)
 - è una tecnica per convertire i dati dal formato usato nel DB in un formato utilizzabile nel linguaggio di programmazione orientato agli oggetti
 - in pratica, una tabella è definita tramite una classe (e i suoi attributi corrispondono a quelli della tabella); un oggetto (istanza di una certa classe) rappresenta una riga della tabella
 - la conversione è eseguita in automatico dal framework
- Nella libreria sqlalchemy, una classe che estende la classe SQLAlchemy è allo stesso tempo:
 - un modello Pydantic (che stiamo già usando)
 - un modello SQLAlchemy (solo se passiamo un parametro specifico *table*)
- Questo ci permetterà di sfruttare la struttura dati che abbiamo usato sinora nelle nostre API per implementare facilmente l'ORM

Modelli Book

- La cosa più veloce sarebbe modificare la classe Book già definita dentro "models/book.py" in modo che estenda un modello SQLAlchemy.
Il parametro `table=True` specifica che si tratta di un modello relazionale (oltre che di un modello Pydantic)

```
from sqlalchemy import SQLAlchemy, Field
```

```
class Book(SQLAlchemy, table=True):  
    id: int = Field(default=None, primary_key=True)  
    # continua uguale...
```

- Ma c'è un problema: settando `table=True` non viene effettuata la validazione dei dati durante la creazione dell'oggetto...
 - ...cosa fondamentale per le nostre API!



Modelli Book

- Questo comportamento è una scelta voluta (anche se molti chiedono che venga modificato...) per non interferire con alcune funzionalità di SQLAlchemy
- Nella pratica però, le strutture dati relative al DB e gli schemi delle API non sono quasi mai uguali...
 - ...anzi, solitamente abbiamo bisogno di diverse "varianti" per la stessa struttura dati!
 - in questi casi, l'uso di modelli SQLAlchemy/Pydantic ci garantisce flessibilità e limita il codice da riscrivere
- Modifichiamo una specifica di progetto iniziale per vedere nel concreto questi vantaggi:
l'ID di un nuovo libro non sarà più scelto dal client, ma assegnato internamente dal DB

Modelli Book

- Avremo bisogno quindi di 3 strutture dati:
 - un modello relazionale per il database **Book** (contenente ID opzionale, titolo, autore, review)
 - un modello per la creazione di un libro **BookCreate** (contenente titolo, autore, review)
 - un modello per restituire nelle API tutti gli attributi di un libro **BookPublic** (contenente ID, titolo, autore, review)
- Dato che le differenze tra questi modelli sono minime, sfruttiamo l'ereditarietà delle classi per minimizzare il codice!
- Per convertire un modello "normale" in un modello relazionale, esiste un apposito metodo di classe:
`Book.model_validate(modello)`
- Modifichiamo il file "models/book.py" (codice nella prossima slide)

Modelli Book

```
from sqlmodel import SQLModel, Field
from typing import Annotated

class BookBase(SQLModel): # superclasse con attributi comuni
    title: str
    author: str
    review: Annotated[int | None, Field(ge=1, le=5)] = None

class Book(BookBase, table=True): # modello relazionale
    id: int = Field(default=None, primary_key=True)

class BookCreate(BookBase): # schema per creare un nuovo libro
    pass

class BookPublic(BookBase): # schema per restituire le info di un libro
    id: int
```



Dependencies

- Dobbiamo adesso modificare tutte le API...
- Prima però, dobbiamo fare in modo di creare, per ogni richiesta in cui è necessario, una sessione con il DB separata. Sfruttiamo le *dependencies* di FastAPI.
- Le dependencies sono *callable* che possiamo inserire nelle funzioni endpoint, e verranno invocate non appena viene ricevuta una richiesta su quell'endpoint. Possono anche ricevere uno o più parametri tra quelli della funzione endpoint (li recuperano in base al loro nome). Si inseriscono così:

```
@app.get("/")  
def get(variable: Annotated[type, Depends(callable)]):  
    pass
```

- A ogni richiesta GET, verrà chiamata "callable", e il risultato (di tipo type) sarà messo dentro "variable" e accessibile all'interno della funzione

Dependencies

- Definiamo dentro il file "db.py" la funzione, che restituirà una nuova istanza di DB ogni volta che verrà chiamata e la terrà aperta per tutto il tempo necessario (grazie all'uso di `yield`)

```
def get_session():  
    with Session(engine) as session:  
        yield session
```

- Per non dover riscrivere in ogni funzione endpoint la sintassi per inserire la dependency, la scriviamo una volta sola e la mettiamo in una variabile (sempre nel file "db.py")

```
SessionDep = Annotated[Session, Depends(get_session)]
```

Modifica API GET /book_list

- Adesso possiamo, in ogni funzione endpoint in cui serve interagire con il DB, aggiungere la dependency. Iniziamo a modificare le API nel file "routers/frontend.py"

```
from data.db import SessionDep  
from sqlmodel import select
```

```
@router.get("/book_list", response_class=HTMLResponse)  
def show_book_list(request: Request, session: SessionDep):  
    books = session.exec(select(Book)).all()  
    # continua uguale...
```

- In questo caso stiamo usando le funzioni della libreria per eseguire una query sulla tabella Book (avremmo anche potuto scrivere la query in SQL "classico" ed eseguirla)



Modifica API GET /books

- Possiamo ora al file "routers/books.py"

```
from models.book import Book, BookCreate, BookPublic
from data.db import SessionDep
from sqlmodel import select
```

```
@router.get("/")
def get_all_books(
    session: SessionDep,
    sort: Annotated[bool, Query(description="Sort books by their review")] = False
) -> list[BookPublic]:
    """Returns the list of available books."""
    books = session.exec(select(Book)).all()
    # continua uguale...
```



Modifica API POST /books

```
@router.post("/")
def add_book(session: SessionDep, book: BookCreate):
    """Adds a new book."""
    session.add(Book.model_validate(book))
    session.commit()
    return "Book successfully added"
```

- In questo caso stiamo effettuando un inserimento, e il DB sceglierà automaticamente l'ID del nuovo libro (non dobbiamo quindi più verificare che esista già)
- Ricordiamoci sempre di chiamare la funzione `commit` per rendere effettive le modifiche al DB (altrimenti le perderemmo al termine della sessione)

Modifica API POST /books_form

```
@router.post("_form/")
def add_book_from_form(
    session: SessionDep,
    book: Annotated[BookCreate, Form()]
):
    """Adds a new book."""
    session.add(Book.model_validate(book))
    session.commit()
    return "Book successfully added"
```

- Ricordiamoci anche di eliminare il campo ID dal form (non riceveremmo comunque errori, il campo relativo verrebbe ignorato)

Modifica API DELETE /books

```
from sqlmodel import select, delete
```

```
@router.delete("/")
def delete_all_books(session: SessionDep):
    """Deletes all the stored books."""
    session.exec(delete(Book))
    session.commit()
    return "All books successfully deleted"
```

- Per le prossime API, abbiamo bisogno (oltre che di SELECT) di usare WHERE nella query. Questo è il modo generico per farlo (ma noi ne useremo uno semplificato):

```
statement = select(Book).where(Book.review == 5)
query_result = session.exec(statement) # è un'iteratore, possiamo già usarlo
books = query_result.all() # se vogliamo una lista
```

Modifica API GET /books/{id}

```
@router.get("/{id}")
def get_book_by_id(
    session: SessionDep,
    id: Annotated[int, Path(description="The ID of the book to get")]
) -> BookPublic:
    """Returns the book with the given ID."""
    book = session.get(Book, id)
    if book:
        return book
    else:
        raise HTTPException(status_code=404, detail="Book not found")
```

Modifica API POST /books/{id}/review

```
@router.post("/{id}/review")
def add_review(
    session: SessionDep,
    id: Annotated[int, Path(description="The ID of the book to which add the
review")],
    review: Review
):
    """Adds a review to the book with the given ID."""
    book = session.get(Book, id)
    if not book:
        raise HTTPException(status_code=404, detail="Book not found")
    book.review = review.review
    session.add(book)
    session.commit()
    return "Review successfully added"
```

Modifica API PUT /books/{id}

```
@router.put("/{id}")
def update_book(
    session: SessionDep,
    id: Annotated[int, Path(description="The ID of the book to update")],
    new_book: BookCreate
):
    """Updates the book with the given ID."""
    book = session.get(Book, id)
    if not book:
        raise HTTPException(status_code=404, detail="Book not found")
    book.title = new_book.title
    book.author = new_book.author
    book.review = new_book.review
    session.add(book)
    session.commit()
    return "Book successfully updated"
```



Modifica API DELETE /books/{id}

```
@router.delete("/{id}")
def delete_book(
    session: SessionDep,
    id: Annotated[int, Path(description="The ID of the book to delete")]
):
    """Deletes the book with the given ID."""
    book = session.get(Book, id)
    if not book:
        raise HTTPException(status_code=404, detail="Book not found")
    session.delete(book)
    session.commit()
    return "Book successfully deleted"
```

Riempimento DB con dati fittizi

- In varie circostanze (soprattutto in fase di test) è utile avere dei dati fittizi con cui riempire il DB. Questo si può fare in maniera automatica, ad esempio con la libreria Faker
 - installiamola con il comando **pip install Faker** (ricordiamoci di aggiungere Faker nel file "requirements.txt")
- Modifichiamo la funzione `init_database` nel file "data/db.py" per riempire il DB ma solo se è la prima creazione (controlliamo se il file esiste già...)

```
from faker import Faker
```

```
def init_database() -> None:
    ds_exists = os.path.isfile("app/data/database.db")
    SQLAlchemy.metadata.create_all(engine)
    if not ds_exists:
        f = Faker("it_IT")
        with Session(engine) as session:
            for i in range(10):
                book = Book(title=f.sentence(nb_words=5), author=f.name(),
                           review=f.pyint(1, 5))
                session.add(book)
            session.commit()
```

Nota importante sulla creazione del DB

- Prima di chiamare la funzione `SQLModel.metadata.create_all(engine)` **è sempre necessario importare tutte le classi che definiscono una tabella del DB!**
Altrimenti, le classi non verrebbero definite e la funzione non saprebbe quali tabelle creare...
- Nel nostro caso dobbiamo assicurarci di avere questo import
`from models.book import Book`
a prescindere dal fatto che poi useremo o meno i moduli importati nel codice successivo
- Se l'IDE protesta per "unused import statement", e in generale quando l'IDE mostra dei warning ma noi:
 - sappiamo esattamente perché abbiamo scritto il codice in quel modo
 - siamo sicuri che non crei problemi

Possiamo silenziare i warning aggiungendo un commento `# NOQA` alla fine della linea

```
from models.book import Book # NOQA
```


Relazioni tra tabelle – One-to-Many

- Per mostrare come gestire relazioni tra diverse tabelle, immaginiamo che nel nostro sistema biblioteca ci siano degli utenti che possono prendere in prestito i libri
- Ogni libro sarà quindi associato a un singolo utente, mentre un utente potrà essere associato anche a più di un libro. Questa relazione è chiamata one-to-many
- Prima di tutto, definiamo un modello dati e relazionale per gli utenti:

```
from sqlalchemy import SQLModel, Field
from datetime import date

class BaseUser(SQLModel):
    name: str
    birth_date: date
    city: str

class User(BaseUser, table=True):
    id: int | None = Field(default=None, primary_key=True)

class UserPublic(BaseUser):
    pass
```



Relazioni tra tabelle – One-to-Many

- Creiamo anche un nuovo file "routers/users.py" per poter visualizzare gli utenti:

```
from fastapi import APIRouter
from data.db import SessionDep
from models.user import User, UserPublic
from sqlmodel import select

router = APIRouter(prefix="/users")

@router.get("/")
def get_all_users(session: SessionDep) -> list[UserPublic]:
    """Returns all users."""
    users = session.exec(select(User)).all()
    return users
```

Relazioni tra tabelle – One-to-Many

- Ricordiamoci anche di aggiornare il file "main.py" per includere il nuovo router:

```
from routers import frontend, books, users
```

```
# ...
```

```
# dopo l'ultima linea:
```

```
app.include_router(users.router, tags=["users"])
```

Relazioni tra tabelle – One-to-Many

- Per creare la relazione tra la tabella "book" e la tabella "user", aggiungiamo una chiave esterna (foreign key) nella tabella "book", che referencia la chiave primaria della tabella "user".
- Nel file "models/book.py":

```
class Book(BookBase, table=True):  
    id: Annotated[int, Field(default=None, primary_key=True)]  
    user_id: int | None = Field(default=None, foreign_key="user.id")
```

- NB: la tabella che viene creata nel DB avrà lo stesso nome della classe ma in minuscolo

Relazioni tra tabelle – One-to-Many

- Adesso possiamo modificare il codice per riempire il DB con dati fittizi dentro "data/db.py", in modo che crei degli utenti e li associ a dei libri:

```
from models.user import User

def init_database() -> None:
    ds_exists = os.path.isfile("app/data/database.db")
    SQLAlchemyModel.metadata.create_all(engine)
    if not ds_exists:
        f = Faker("it_IT")
        with Session(engine) as session:
            for i in range(10):
                user = User(
                    name=f.name(), birth_date=f.date_of_birth(), city=f.city()
                )
                session.add(user)
            session.commit()
            for i in range(10):
                book = Book(
                    title=f.sentence(nb_words=5), author=f.name(),
                    review=f.pyint(1, 5),
                    user_id=f.pyint(1, 10)
                )
                session.add(book)
            session.commit()
```

Relazioni tra tabelle – One-to-Many

- Infine, aggiungiamo un endpoint dentro "routers/users.py" per fare JOIN delle tabelle e mostrare i libri presi in prestito da un utente:

```
from models.book import Book, BookPublic

@router.get("/{id}/books")
def get_user_books(
    id: int,
    session: SessionDep
) -> list[BookPublic]:
    """Returns all books held by the given user."""
    statement = select(Book).join(User).where(User.id == id)
    result = session.exec(statement).all()
    return result
```



Relazioni tra tabelle – Many-to-Many

- Immaginiamo adesso di avere più copie di ogni libro (senza preoccuparci di *quante* siano); quindi abbiamo bisogno di associare più libri a più utenti, e viceversa
- Questa relazione è chiamata many-to-many, e per gestirla necessita di una terza tabella che contiene (come chiavi esterne) le chiavi primarie delle due tabelle che vogliamo mettere in relazione.
- Iniziamo eliminando la foreign key dalla classe Book dentro "models/book.py":

```
class Book(BookBase, table=True):  
    id: Annotated[int, Field(default=None, primary_key=True)]
```

Relazioni tra tabelle – Many-to-Many

- Creiamo poi un nuovo file "models/book_user_link.py":

```
from sqlalchemy import SQLAlchemy, Field
```

```
class BookUserLink(SQLAlchemy, table=True):  
    book_id: int = Field(foreign_key="book.id", primary_key=True)  
    user_id: int = Field(foreign_key="user.id", primary_key=True)
```

- Il campo `primary_key=True` non è obbligatorio, serve solo per evitare righe duplicate.

Relazioni tra tabelle – Many-to-Many

- Modifichiamo l'endpoint `/users/{id}/books`

```
from models.book_user_link import BookUserLink

@router.get("/{id}/books")
def get_user_books(
    id: int,
    session: SessionDep
) -> list[BookPublic]:
    """Returns all books held by the given user."""
    statement = select(Book).join(BookUserLink).where(
        BookUserLink.user_id == id
    )
    result = session.exec(statement).all()
    return result
```

Relazioni tra tabelle – Many-to-Many

- Infine, il codice per riempire il DB con dati fittizi dentro "data/db.py":

```
from models.book_user_link import BookUserLink

def init_database() -> None:
    ds_exists = os.path.isfile("app/data/database.db")
    SQLAlchemyModel.metadata.create_all(engine)
    if not ds_exists:
        f = Faker("it_IT")
        with Session(engine) as session:
            for i in range(10):
                user = User(
                    name=f.name(), birth_date=f.date_of_birth(), city=f.city())
                session.add(user)
            session.commit()
            for i in range(10):
                book = Book(title=f.sentence(nb_words=5), author=f.name(),
                           review=f.pyint(1, 5))
                session.add(book)
            session.commit()
            for i in range(10):
                link = BookUserLink(book_id=f.pyint(1, 10), user_id=f.pyint(1, 10))
                session.add(link)
            session.commit()
```